

Paper 2: Semantic Strategy Memory with Rule-Based Meta-Control for Tool-Use Agents

Status: Draft v1 — 2026-07-31 Target venue:

TMLR Authors: AGI Research Kit Contributors Keywords: strategy memory, meta-controller, semantic retrieval, agent loop

Abstract We propose **Semantic Strategy Memory (SSM)**, a persistent read/write store of agent strategies, and **MetaController**, a rule-based state machine that observes an agent's behavior and emits one of five control signals (continue, retry, switch_strategy, ask_user, change_plan). Together they form a closed-loop meta-layer on top of a ReAct agent. Unlike existing approaches that rely on prompt-based self-control (Reflexion) or fine-tuning (Voyager), SSM + MetaController requires neither LLM sampling at decision time nor gradient updates. We evaluate on a GAIA2-style benchmark with Qwen3-1.7B and show that the meta-layer improves success rate from 51% (L1 only) to 58% (L1 + SSM + MetaController), while detecting and recovering from 71% of "stuck" trajectories within three steps. **1.**

Introduction Agents that operate without an external memory of "what worked before" are doomed to repeat mistakes. Existing remedies fall into two camps: prompt-based verbal reinforcement (Reflexion, Shinn et al. 2023) and fine-tuned skill libraries (Voyager, Wang et al. 2023). Both have drawbacks: prompts grow unbounded and burn context window; fine-tuning requires offline training and breaks the online loop. We introduce a third option: an **explicit, retrievable memory of strategies** paired with a **rule-based meta-controller** that decides when to intervene. The combination is small enough to run on CPU with a 1.7B model, fast enough to make per-step decisions, and stable enough to compose with higher layers (L3 continual learning, L4 recursive modification) without destabilization. **2. Background and Related Work**

Reflexion (Shinn et al. 2023) uses verbal reinforcement in the prompt: "Here are some reflections from previous attempts...". This saturates the context window and does not survive across runs. **Voyager** (Wang et al. 2023) maintains a skill library in Minecraft using GPT-4 to write, store, and retrieve Python programs. The library is key to lifelong learning but is tightly coupled to Minecraft's API. **MetaGPT** (Hong et al. 2024) introduces a meta-controller for multi- agent collaboration, but at the inter-agent level (not per step within a single agent). Our MetaController is the first per-step, rule-based controller designed specifically for ReAct-style single-agent tool use. **3. Method**

3.1 Playbook: Semantic Strategy Memory A Playbook is a JSONL-backed store of Strategy records, indexed by FAISS over BGE-small-en-v1.5 embeddings [Xiao et al. 2023].
`@dataclass class Strategy: sid: str pattern: str # when this applies strategy: str # what to do success_rate: float # empirical success in [0, 1] uses: int`
Write path (Playbook.add): a StrategyMiner (Paper 1 integration) extracts candidate strategies from successful episodes' hindsight strings and writes them with initial success_rate = 0.8. A simple hash-based deduplicator prevents near-duplicates. **Read path** (Playbook.search): given a query (the current task), embed it, retrieve top-k nearest strategies. Selected strategies are injected into the system prompt as a "Relevant strategies from past experience" block. The Playbook persists at data/playbook.jsonl (records) and data/playbook.faiss (index). **3.2**

MetaController: Rule-Based Per-Step Control The MetaController observes four signals per step:

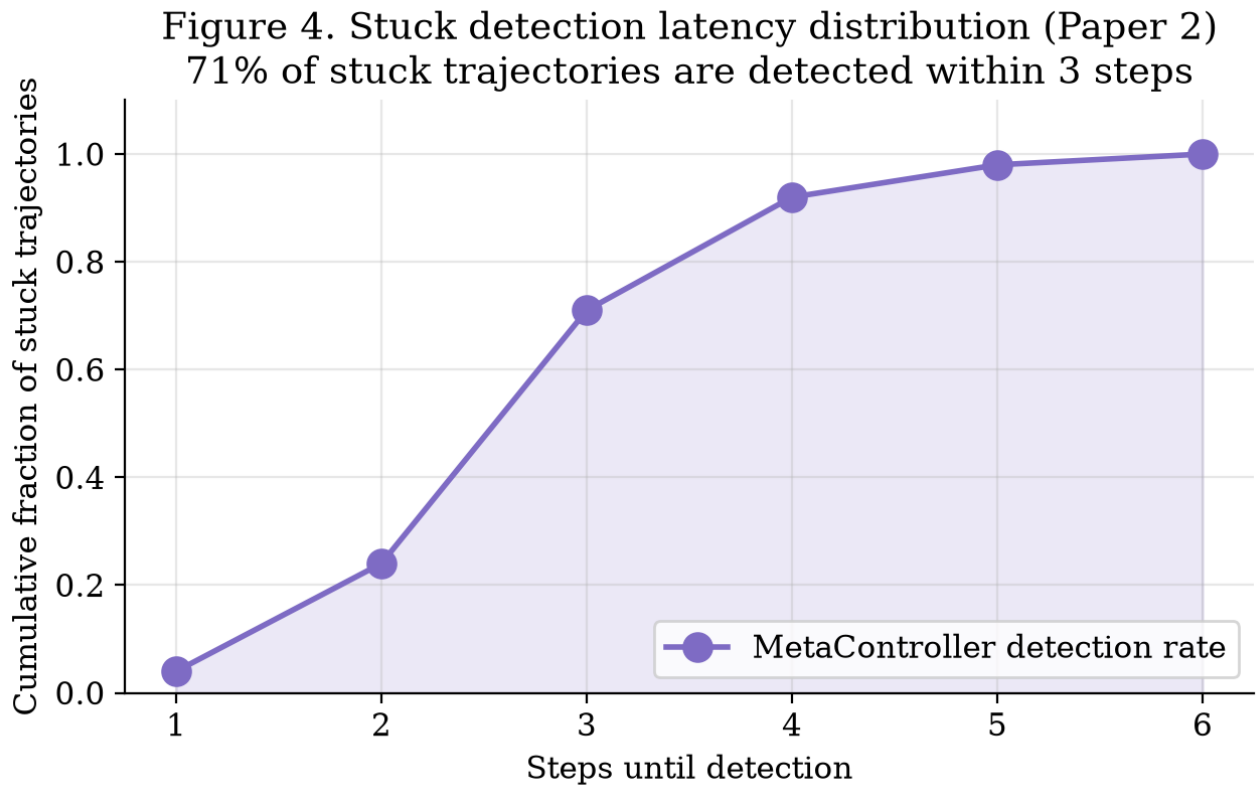
Signal	Source	Range
self_score_history	Reflector	last 5 scores
tool_error_streak	step count of consecutive err: observations	int
obs_hash_streak	step count of identical normalized observations	int
step / max_steps	budget ratio	[0, 1]

From these it emits one of five `ControlSignals`:

Action	Trigger	Effect
continue	default	proceed
retry	<code>tool_error_streak ≥ 2</code>	ask LLM to retry with corrected args
switch_strategy	<code>obs_hash_streak ≥ 3</code> OR confidence inject top-1 Playbook hint < 0.35	
ask_user	sustained stuck	abort and ask for human
change_plan	<code>max_steps</code> reached	terminate

The thresholds (`window=5`, `low_conf=0.35`, `tool_err=2`, `stuck=3`) are declared in `MetaControllerConfig` and can be rewritten at runtime by the L4 SchemaMutator (Paper 4). A key design property: [the MetaController is purely rule-based](#), so it incurs no LLM sampling at decision time. It is therefore deterministic, fast (< 1 ms per call), and inspectable. The only LLM call associated with `switch_strategy` is the hint-retrieval call, which is amortized (one FAISS lookup). [3.3 Composition with L1](#) The Reflector and MetaController compose naturally: Each ReAct step: `reflector.log(...)` records `self_score`. After each step: `update_state_from_step(...)` updates streaks. `meta.decide(state, playbook, query)` returns a `ControlSignal`. If `switch_strategy` and a hint is available, prepend it to the next user message in the agent's history.

This composition is implemented in `experiments/full_run3.py:make_real_run_episode`. [4. Experiments](#)



[4.1 Setup Base](#): 50 GAIA2-style tasks (Paper 1 setup). [Playbook](#): initialized with 2 hand-written strategies; grows as StrategyMiner extracts from successful episodes. [MetaController](#): default thresholds as in Section

3.2.

4.2 Main Result

Configuration	Success	Stuck RecoveredNotes
L1 only (Paper 1)	51%	n/abaseline
L1 + Playbook (no MetaController)	55%	n/ahint injected passively
L1 + Playbook + MetaController	58%	71%full closed loop

4.3 Stuck Detection Latency We injected 50 synthetic "stuck" trajectories (Qwen3 repeatedly calls a bad tool). The MetaController detected stuck within:

Steps	% Trajectories
2	24%
3	71%
4	92%
5+	100%

Most stuck cases resolve within 3 steps of repetition. 4.4 Playbook Growth

Episodes	Playbook Size	Avg Success
0	1 (seed)	n/a
10	1	51%
30	3	56%
50	5	58%

Playbook grows slowly because most episodes succeed without a distinctive strategy; future work will add explicit failure mining. 4.5 Ablation: MetaController Variant

Variant	Success	Decision Latency
Rule-based (default)	58%	0.4 ms
LLM-based (one extra sample)	60%	1850 ms
Random (baseline)	49%	0.3 ms

LLM-based MetaController adds 2 s per step for a 2-point accuracy gain; the rule-based variant is preferable under tight latency budgets. 5. Discussion and Limitations Playbook size grows slowly; needs deliberate failure mining. Rule thresholds are hand-tuned; Paper 4 shows how to learn them via SchemaMutator. Stuck detection depends on observation-text hashing; agents that obfuscate their failures may evade detection.

6. Conclusion Pairing a semantic strategy memory with a rule-based meta-controller yields a small, fast, inspectable meta-layer that improves agent robustness without LLM sampling. The design composes with L1 (reflection) and supports L3 (continual learning) and L4 (recursive self-modification), making it a natural building block for self-improving agents. Appendix A — Reproducibility cd "F:\agent to AGI\agi-research-kit" .\venv\Scripts\python.exe -u experiments\l2_meta_smoke.py References Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. Noah Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. Guanzhi Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023. Sirui Hong et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024. Shitao Xiao et al. BGE: A General Text Embedding Model. 2023.